

WEEK 4 (Praveen 2520030406)

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("fork() failed\n");
        exit(1);
    }

    else if (pid == 0)
    {
        // Child process
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Child is terminating...\n");

        exit(0);
    }

    else
    {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is sleeping...\n");

        sleep(30);

        printf("Parent is terminating...\n");
    }

    return 0;
}
```

Output:

```
[praveenbusala@Praveens-MacBook-Air-2 CLG % nano week3.c
[praveenbusala@Praveens-MacBook-Air-2 CLG % gcc week3.c
[praveenbusala@Praveens-MacBook-Air-2 CLG % ./a.out
Parent Process Started
Parent PID: 24094

--- Parent Process ---
Parent PID: 24094
Child PID: 24095
Parent is waiting for child...

--- Child Process ---
Child PID: 24095
Child PPID: 24094
Child is running...
Child is terminating...
Child terminated.
Parent process terminating...
praveenbusala@Praveens-MacBook-Air-2 CLG %
```